

CH1

Performance 效能 的兩大指標

- Response time: How Long
- Throughput: How many work per unit

可以影響 response time 與 throughput:

- Switch to a faster processor
- Adding more processor

Performance (效能) = $1/\text{Execution Time}$ (執行時間)

Accurately measuring the execution time of a program is not easy:
因為同時有很多 process 在跑, 會互相影響

Cram school time

$$\text{Program Performance (程式效能)} = \frac{1}{\text{Execution Time}}$$

$$\text{CPU Performance} = \frac{\text{Instructions}}{\text{Seconds}}$$

Clock Cycle Time: CPU 每個時鐘的週期

Clock Rate: CPU 的 Hertz 數 -> a.k.a CPU Frequency

CPU (Execution) Time: CPU 執行一個程式的耗時

Instruction Count: CPU執行一個 program 的指令 (32bit 那種) 數量

Clock Cycles: CPU執行一個 Program 的時鐘週期數量 -> Clock Period

Cycles Per Instruction (CPI): '平均'執行一個指令的時鐘週期數量

Formula

$$\text{CPU Performance} = \frac{1}{\text{CPI} \times \text{Clock Cycle Time (Period)}} = \frac{\text{Clock Rate (Freq)}}{\text{CPI}}$$

$$\text{Clock Cycle Time} = \frac{1}{\text{Clock Rate}}$$

$$\text{CPU Time} = \text{Clock Cycles} \times \text{Clock Cycle Time} = \frac{\text{Clock Cycles}}{\text{Clock Rate}}$$

$$\text{Clock Cycles} = \text{CPU Time} \times \text{Clock Rate}$$

$$\text{CPI} = \frac{\text{Clock Cycles}}{\text{Instruction Count}} = \frac{\text{CPU Time} \times \text{Clock Rate}}{\text{Instruction Count}}$$

How much faster = 算比例

$$\text{Capacitive loads} = \frac{\text{Dynamic Power}}{(\text{Voltage})^2 \times \text{Clock Rate}}$$

Dissipated (消耗) power = Static Power + Dynamic Power

阿姆達爾定律 Amdahl's Law

$$F = \frac{\text{Time that can be improve}}{\text{Total Time}} \quad \text{improve 佔總時間的比率}$$

$$S = \frac{\text{Improve Time Before}}{\text{Improve Time After}} \quad \text{可 improve 時間的比率}$$

$$\text{Speed Up} = \frac{\text{Original Time}}{\text{Improved Time}} = \frac{\text{Original Time}}{\text{Improveable Time} + \text{Unimproveable Time}} = \frac{1}{(1 - F) + F/S}$$

Q-1.8 A processor which $S_{old} = 10$, $D_{old} = 90$, $V_{old} = 1.25$

(a) Find the percentage of the total dissipated (消耗) power comprised by static power and the ratio of static power to dynamic power for each technology.

名詞解說

FP Instruction: 浮點數計算

INT Instruction: 整數計算

L/S Instruction: Load and Store 例如 if else 判斷

Arithmetic : 算數運算

R-format: 針對暫存器 (Register) 的格式

Q-1.8 If the total dissipated power is to be reduced by 10%, how much should the voltage be reduced to maintain the same leakage current?

Note: power is defined as the product of voltage and current.

$$\frac{\text{Static Power}_{\text{new}} + \text{Dynamic Power}_{\text{new}}}{\text{Static Power}_{\text{old}} + \text{Dynamic Power}_{\text{old}}} = 0.9$$

$$\text{Dynamic Power} = C \times V^2 \times \text{Frequency}$$

$$\text{Static Power} = V \times I_{\text{leak}}$$

$$S_{\text{new}} = V_{\text{new}} \times (S_{\text{old}} / V_{\text{old}})$$

$$D_{\text{new}} = 0.9 \times (S_{\text{old}} + D_{\text{old}}) - S_{\text{new}}$$

$$V_{\text{new}} = \sqrt{D_{\text{new}} / C \times F}$$

$$\frac{V_{\text{new}} \times I}{100} + C \times V^2 \times F = 90$$

$$S_{\text{new}} = V_{\text{new}} \times (10 / 1.25)$$

$$D_{\text{new}} = 0.9 \times (10 + 90) - V_{\text{new}} \times (10 / 1.25)$$

Reducing Power

□ Suppose a new CPU has

- 85% of capacitive load of old CPU
- 15% voltage and 15% frequency reduction

$$\frac{P_{\text{new}}}{P_{\text{old}}} = \frac{C_{\text{old}} \times 0.85 \times (V_{\text{old}} \times 0.85)^2 \times F_{\text{old}} \times 0.85}{C_{\text{old}} \times V_{\text{old}}^2 \times F_{\text{old}}} = 0.85^4 = 0.52$$

□ The power wall

CMOS技術，難以進一步降低電壓，必須尋求其他技術改善效能

- We can't reduce voltage further
- We can't remove more heat

□ How else can we improve performance?

忽略static power

必考

Exercise

□ X公司電腦

- 記憶體讀取時間，佔電腦運算20%時間
- 處理器運算時間，佔電腦運算60%時間
- 如何決定開發什麼？

□ 目前有兩項技術，如何決定優先開發哪一項？

- 1. 將記憶體讀取速度提高到原來的5倍
- 2. 將處理器速度提高到原來150%

$$\text{○ } S1 = 1 / ((1 - 20\%) + 20\% / 5) = 1.19$$

$$\text{○ } S2 = 1 / ((1 - 60\%) + 60\% / 1.5) = 1.25$$

proportional 比例

□ Example: 程式執行須100 s，其中乘法(multiply)運算佔 80 s

- 若程式執行要達原來5倍 (100/5=20 s)，multiply運算要提高到原來幾倍? (Amdahl's Law)

$$\begin{aligned} & \text{Execution time after improvement} \\ &= \frac{\text{Execution time affected by improvement}}{\text{Amount of improvement}} + \text{Execution time unaffected} \end{aligned}$$

$$\text{Execution time after improvement} = \frac{80 \text{ seconds}}{n} + (100 - 80)$$

$$20 = \frac{80}{n} + 20$$

n=∞, Can't be done!

- Corollary: make the common case fast (將資源花在最常使用案例)³⁵

Metric 度量

.text	# Define the program instructions.
main:	# Label to define the main program.
li \$v0,4	# Load 4 into \$v0 to indicate a print string.
la \$a0, greeting	# Load the address of the greeting into \$a0.
syscall	# Print greeting. The print is indicated by
	# \$v0 having a value of 4, and the string to
	# print is stored at the address in \$a0.
li \$v0, 10	# Load a 10 (halt) into \$v0.
syscall	# The program ends.
.data	# Define the program data.
greeting: .asciiz "Hello World"	#The string to print.

\$v0 = 中斷暫存器

\$a0 = 位址暫存器

segment (區段): 在 .text .data .asciiz 下為一個區段

li: Load the immediate (即時) value into register

la: Load the address at the lable into the register

syscall: Request a system services

Immediate value: A constant included in the instruction

中斷類型：

id	1	4	5	10
Function	Print Int	Print Str	Input	Halt

MIPS Commands:

lw \$s1, 12(\$t0): Load Word (A Byte) from 12(\$t0) to \$s1

12(\$t0): is a memory address calculate from the base \$t0 + 12 (like: A[12])

li \$s2, 5: Load immediate (constant) value 5 to \$s2

add \$s3, \$s1, \$s2: \$s3 = \$s1 + \$s2

sw \$s3, 12(\$s4) : Store word

CPU 除了以上 32 個 registers 外還有：

No.33: PC (Program Counter)

No.34: EPC (Exceptoin Process Counter)

MIPS Commands:

R – Format :

OP (5bit)	\$rs (bit)	\$rt (bit)	\$rd (bit)	sa	funct
-----------	------------	------------	------------	----	-------

Example: add, \$rd, \$rs, \$rt

I – Format :

OP (5bit)	\$rs (5bit)	\$rt (5bit)	Immediate (17bit)
-----------	-------------	-------------	-------------------

Example: li \$rs <immediate/constant>(Will be convert to I-Instruction by assembler)

```
li $t0, 1234           # Pseudo-instruction
addi $t0, $zero, 1234  # Equivalent to
```

J – Format :

OP (5bit)	Jump Target (27bit)
-----------	---------------------

Example: ? ?

\$rs (R-Source), \$rt (R-Target), \$rd (R-Destination)

計算機組織 Zuvio #1

章工二

1135900 21

邱冠勳

1. \$0: 零常數

\$1: 為編譯器保留

\$2: 表達式評估

\$3: 函式結果

\$4: { 函式參數傳入
 ⋮
 \$7: { (超過4個傳 array ptr)

\$8: { 暫時性暫存器
 ⋮
 \$15: { caller saves, callee can clobber
 t0~t7

\$16: { 呼叫者存檔
 ⋮
 \$23: { caller can clobber
 s0~s7

\$24: { 暫時性暫存器
 \$25: { (continue'd)

\$26: { 為系統內核保留
 \$27: { k0 k1

\$28 gp 指標: 指向全域變數

\$29 sp 指標: 指向堆疊的頭

\$30 fp frame pointer

\$31 ra return address.

2. 你好世界註解

```
.text          # 區段開始
main:          # Label Name
    li $v0, 4    # 將中斷ID 4 移入中斷 register
    la $a0, greeting # 將 greeting 的記憶體 ADDR 存入 $a0
    syscall      # 呼叫系統服務 (輸出)

    li $v0, 10   # 將中斷ID 10 移入中斷 register
    syscall      # 呼叫系統服務 (結束程式)

.data          # 區段開始
greeting: .asciz "Hello World" # 字串指標
```